

Déploiement d'une Application Microservices avec Docker & Kubernetes

Soutenance de TP/Projet — NAT MIA4 28. IPSSI

Steven JANSEN & Julien CANTAU · 10/07/2026



INTRODUCTION

Objectifs du Projet

Containeriser et orchestrer une application de type **forum** (posts, commentaires, modération) composée de **6 microservices**, en exploitant Docker pour l'isolation et Kubernetes pour l'orchestration.

Containerisation

Un Dockerfile par service
basé sur node:alpine

Orchestration

Deployments, Services et
Ingress Kubernetes

Communication

Architecture asynchrone via un event-bus central



ÉQUIPE

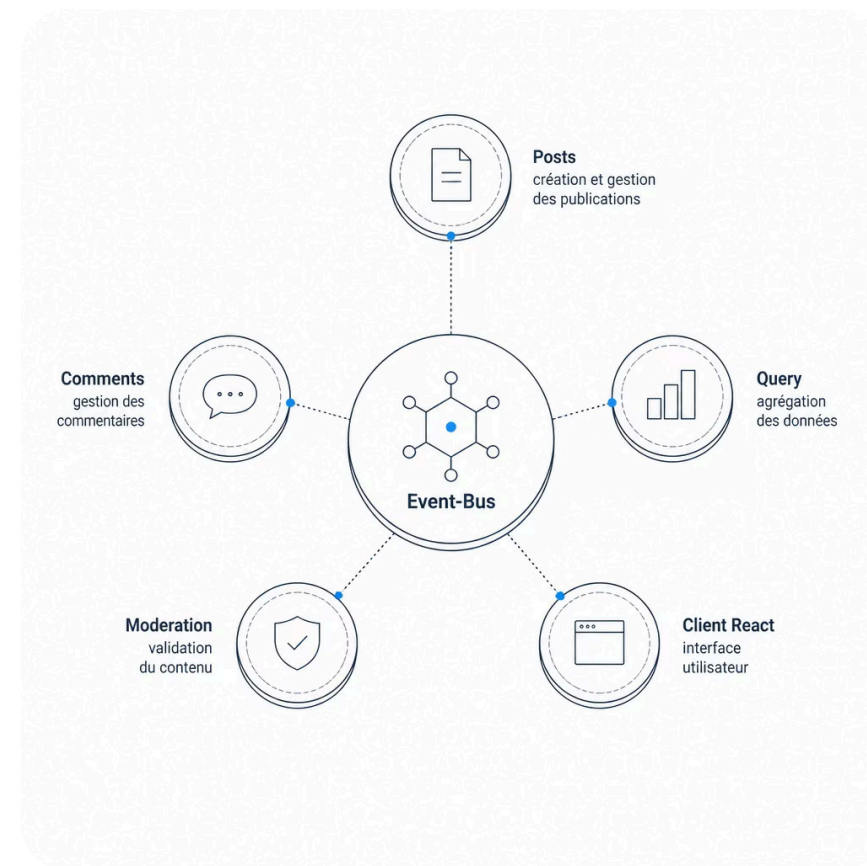
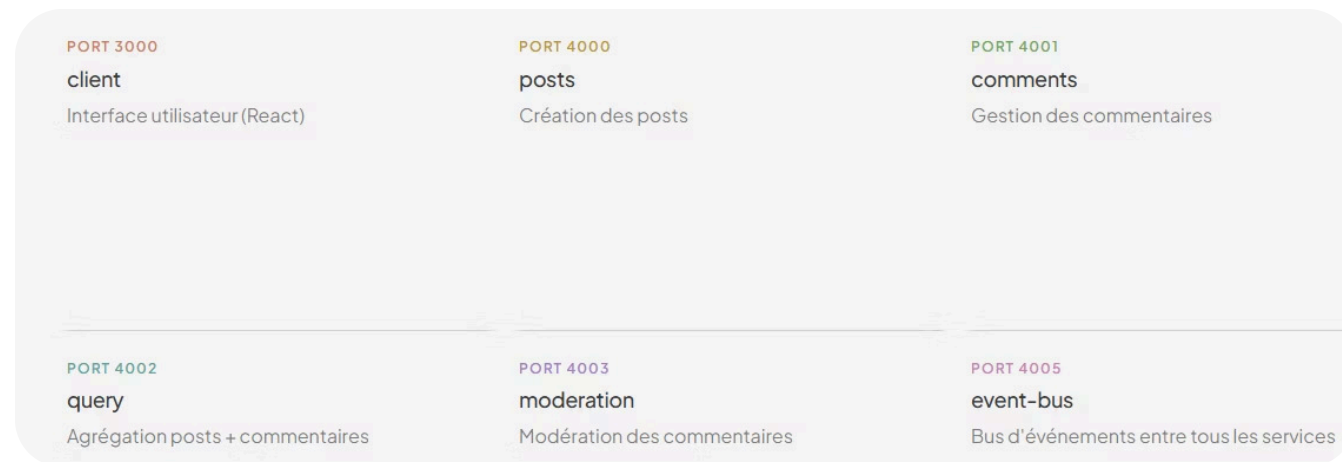
Répartition du Travail

Une répartition complémentaire et solidaire.



Les 6 Microservices

Chaque service est indépendant, conteneurisé et communique via un **event-bus central** garantissant le découplage et la scalabilité de l'ensemble.



Le Dockerfile

Un patron identique pour chaque microservice, optimisé pour la vitesse de build.

Image de base légère

Image `node:alpine` (~100 Mo) image minimale, temps de téléchargement réduit et surface d'attaque limitée

Cache Docker optimisé

CCCOPY package.json puis npm install avant le code source, les dépendances sont mises en cache tant que package.json ne change pas.

Isolation totale

Chaque service s'exécute dans son propre conteneur, sans dépendance directe sur l'environnement hôte.

.dockerignore par service

Exclut `node_modules` du contexte de build — évite d'envoyer des milliers de fichiers inutiles au daemon Docker.

Construction, tests locaux et publication Docker Hub

1

Build

6 images construites.

2

Test local

Utilisation de docker logs pour debug

3

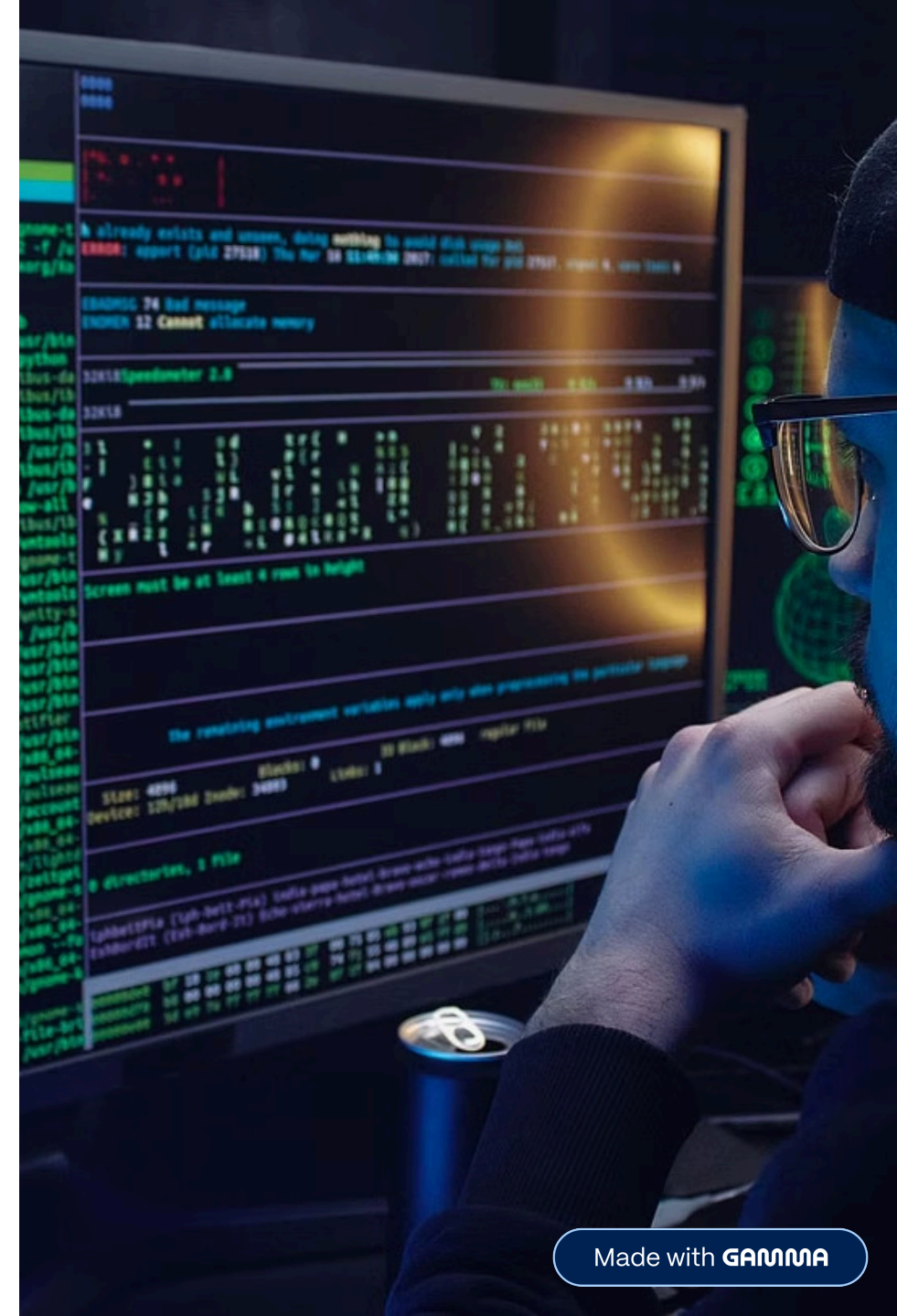
Publish

Tag et Push vers Docker Hub.

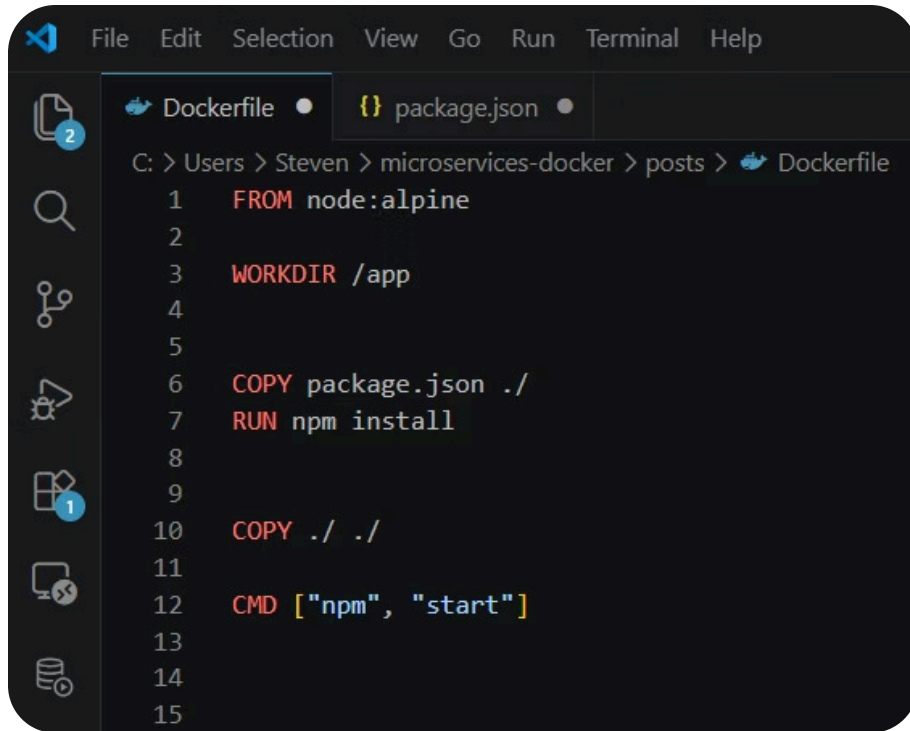
4

Limites

Pas de communication inter-services.
Orchestration via Kubernetes (Julien).

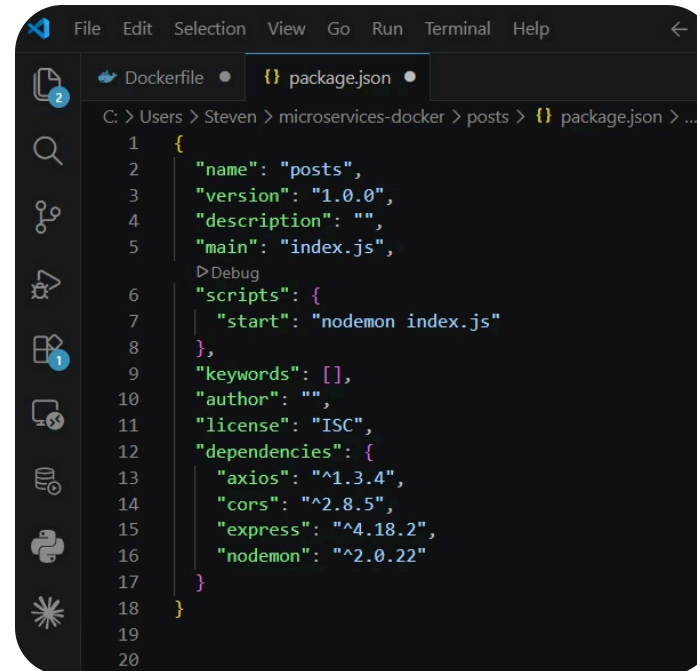


DOCKER



The image shows a VS Code editor window with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The Explorer sidebar on the left shows a file tree with 'Dockerfile' and 'package.json'. The Dockerfile is open in the editor, showing the following content:

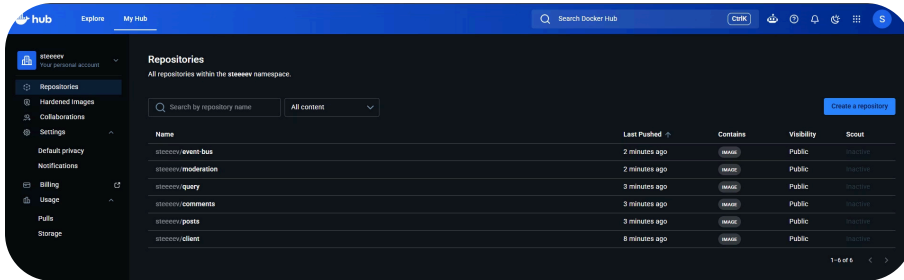
```
C: > Users > Steven > microservices-docker > posts > Dockerfile
1 FROM node:alpine
2
3 WORKDIR /app
4
5
6 COPY package.json ./
7 RUN npm install
8
9
10 COPY ./ ./
11
12 CMD ["npm", "start"]
13
14
15
```



The image shows a VS Code editor window with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The Explorer sidebar on the left shows a file tree with 'Dockerfile' and 'package.json'. The package.json is open in the editor, showing the following content:

```
C: > Users > Steven > microservices-docker > posts > package.json > ...
1 {
2   "name": "posts",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "nodemon index.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "dependencies": {
13    "axios": "^1.3.4",
14    "cors": "^2.8.5",
15    "express": "^4.18.2",
16    "nodemon": "^2.0.22"
17  }
18 }
19
20
```


DOCKER



```
C:\Users\Steven\microservices-docker> docker push steeeev/event-bus:v1
The push refers to repository [docker.io/steeeev/event-bus]
260d099624fa: Pushed
0ed4e7887ea5: Pushed
55afalecc21d: Mounted from steeeev/query
efa380558616: Mounted from steeeev/query
8daabfc0032e: Mounted from steeeev/query
39dee62c4b00: Pushed
fc930c27f781: Mounted from steeeev/query
c56324fea837: Pushed
sha256:f40802721739b55b14436f7346bd51ff4abd68dbaf007feb0942996dfb563e3a size: 856
```

```
docker build -t steeeev/posts:v1 ./posts
docker run -d -p 4000:4000 steeeev/posts:v1
docker push steeeev/posts:v1
```

```
PS C:\Users\Steven> cd C:\Users\Steven\microservices-docker
PS C:\Users\Steven\microservices-docker> docker build -t steeeev/client:v1 ./client
[+] Building 221.4s (11/11) FINISHED
=> [internal] load build definition from Dockerfile                                docker:desktop-linux 0.0s
=> => transferring dockerfile: 342B                                              0.0s
=> [internal] load metadata for docker.io/library/node:alpine                    0.8s
=> [auth] library/node:pull token for registry-1.docker.io                      0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 77B                                                  0.0s
=> [1/5] FROM docker.io/library/node:alpine@sha256:e88a35be04478413b7c71c455cd9865de9b9360e1f43456be5951032d7ac1 0.1s
=> => resolve docker.io/library/node:alpine@sha256:e88a35be04478413b7c71c455cd9865de9b9360e1f43456be5951032d7ac1 0.1s
=> [internal] load build context                                                0.2s
=> => transferring context: 705.07kB                                             0.2s
=> CACHED [2/5] WORKDIR /app                                                    0.0s
=> [3/5] COPY package.json ./                                                  0.1s
=> [4/5] RUN npm install                                                         171.9s
=> [5/5] COPY ./ ./                                                            0.8s
=> exporting to image                                                            47.3s
=> => exporting layers                                                            33.1s
```


Deployments & Services ClusterIP

Deployments

Gèrent les **répliques de pods** et assurent l'**auto-réparation** : si un pod tombe, Kubernetes le recrée automatiquement pour maintenir le niveau de service souhaité.

Services ClusterIP

Attribuent à chaque microservice un **nom DNS stable** au sein du cluster. Les services communiquent entre eux via ce nom, sans connaître les adresses IP dynamiques des pods.

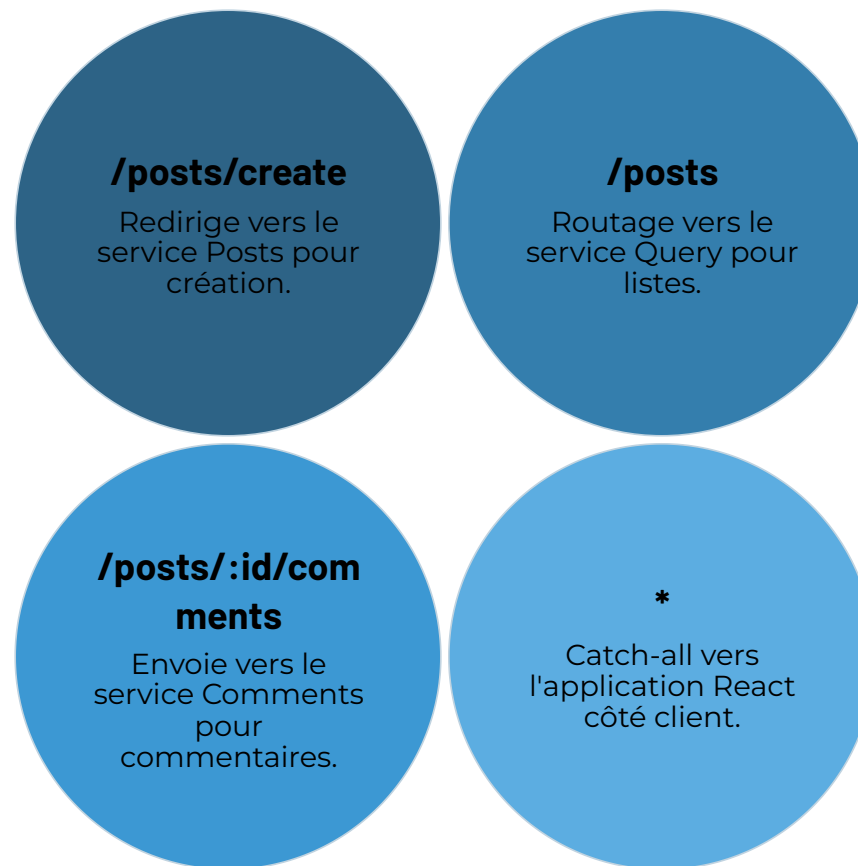
Noms des Services

client-srv

- posts-clusterip-srv
- comments-srv
- query-srv
- moderation-srv
- event-bus-srv

Le Rôle de l'Ingress

L'Ingress centralise le **point d'entrée HTTP unique** et route les requêtes vers le bon service selon le chemin d'URL, évitant d'exposer chaque service individuellement.



Un seul point d'exposition (`localhost`) pour l'ensemble de l'application, avec un routage transparent géré par l'Ingress Controller.

Déploiement et vérification

Appliquer les manifestes

kubectl apply -f k8s/ déploie l'ensemble des ressources du cluster

Vérifier les ressources

kubectl get pods / svc / ingress — état des pods, Services et Ingress

Contrôler les logs

kubectl logs inspecte la sortie de chaque conteneur en cours d'exécution

Diagnostiquer les erreurs

ImagePullBackOff : tag incorrect ou image absente du registre

DÉMONSTRATION

Démo Live



Difficultés Rencontrées

- 1 Dockerfile ignoré par Git**
Cause : Le .gitignore initial excluait les fichiers Dockerfile, qui n'étaient donc jamais poussés sur GitHub malgré le git add
- 2 ImagePullBackOff au premier déploiement**
Cause : Le tag de l'image dans le Deployment ne correspondait pas exactement à celui poussé sur Docker Hub
- 3 Périmètre du projet mal cerné au départ**
Cause : On avait modifier le code de l'application elle-même (React, logique des services) en pensant devoir l'adapter

Bilan & Perspectives

Ce que nous avons appris

- Containerisation avec Docker et optimisation des images (node:alpine, cache des layers)
- Orchestration Kubernetes : Deployments, Services, DNS interne
- Routage HTTP centralisé via l'Ingress
- Architecture microservices et communication asynchrone via un event-bus
- La différence entre le code source (GitHub) et l'image exécutable prête à être déployée (Docker Hub)

Pistes d'amélioration

- Persistance des données
- Pipeline CI/CD
- Sécurisation